PROJECT FOR PRODUCT DEVELOPMENT REPORT

on

FORMAL VERIFICATION OF SECURE NETWORK SLICING – DEFINED NETWORKS

Submitted in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering

6th Semester Submitted by

1. Manisha Behera (2301109139)
2. Abhijit Samal (2301109096)
3. Dibya Jyoti Pradhan (2301109125)
4. P Durga prasad (2301109149)
5. Anshuman Bal (2301109106)

Under the guidance of

Dr. Mitali Sinha
Mrs. Chapala Maharana

Department of Computer Science & Engineering

Parala Maharaja Engineering College, Berhampur, Odisha

Academic Session: 2025-26

CERTIFICATE

This is to certify that the project entitled " FORMAL VERIFICATION OF SECURE NETWORK SLICING IN SOFTWARE-DEFINED NETWORKS" is a Bonafide record of the work carried out by Manisha Behera (2301109139), Abhijit Samal (2301109096), Dibya Jyoti Pradhan (2301109125), P Durga prasad (2301109149), Anshuman Bal (2301109106) students of the Department of Computer Science and Engineering, Parala Maharaja Engineering College, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (B.Tech) in Computer Science and Engineering during the academic year 2025–2026.

This project work has been carried out under our supervision and guidance. To the best of our knowledge, this work is original and has not been submitted previously, either in part or in full, for the award of any other degree or diploma in any institution.The students have demonstrated satisfactory progress and have successfully completed the project work as per the academic guidelines. The project report is approved as it meets the required standards and fulfils all the necessary academic requirements.

Dr. Mitali Sinha
Assistant Professor

Mrs. Chapala Maharana
Assistant Professor

Dr. Sasmita Rani Behera
Associate Professor

Signature of Mentors

Signature of HOD

Date:

Place:

DECLARATION

We hereby declare that the project report entitled **“ FORMAL VERIFICATION OF SECURE NETWORK SLICING IN SOFTWARE-DEFINED NETWORKS ”**
submitted by us to the Department of Computer Science and Engineering, **Parala Maharaja Engineering College**, is a genuine record of our original work carried out by us under the guidance and supervision of Mrs. Mitali Sinha and Mrs. Chapala Maharana in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B. Tech)** in Computer Science and Engineering during the academic year **2025–2026**.

We further declare that this project work has not been submitted, either in full or in part, to any other university or institution for the award of any degree, diploma, or certificate. The information, data, and references used in this report have been acknowledged properly wherever applicable.

We understand that if any part of this declaration is found to be false, incorrect, or misleading, the institution has the right to take appropriate action as per academic rules and regulations.

Signature of students:

1. Manisha Behera (2301109139)
2. Abhijit Samal (2301109096)
3. Dibya Jyoti Pradhan (2301109125)
4. P Durga prasad (2301109149)
5. Anshuman Bal (2301109106)

Date: Place:

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have contributed directly or indirectly to the successful completion of our project titled “FORMAL VERIFICATION OF SECURE NETWORK SLICING -DEFINED NETWORKS”

First and foremost, we would like to extend our heartfelt thanks to our respected project guides, Dr. Mitali Sinha and Mrs. Chapala Maharana, for their constant guidance, valuable suggestions, and continuous encouragement throughout the development of this project. Their expertise and support played a crucial role in shaping our ideas and improving the quality of our work.

We are deeply thankful to the Head of the Department, Computer Science and Engineering, Parala Maharaja Engineering College, for providing us with the necessary resources, infrastructure, and a motivating academic environment that enabled us to complete this project successfully.

We would also like to express our sincere appreciation to all the faculty members of the Computer Science and Engineering Department for their guidance, knowledge sharing, and support during the course of this project work.

ABSTRACT

Software-Defined Networking (SDN) enhances network flexibility by separating the control plane from the data plane and enabling centralized programmability. However, this architectural shift increases the complexity of network behaviour, making it difficult for traditional testing techniques to explore all possible execution states. As a result, errors such as packet loss, policy violations, or deadlocks may remain undetected. To overcome these challenges, this work adopts a formal verification approach to mathematically guarantee the correctness of packet transmission and system behavior under all possible conditions.

The proposed system models an SDN environment consisting of hosts, switches, and a centralized controller, incorporating network slicing to partition the network into multiple logically isolated segments. Each slice operates independently with its own communication policies and forwarding rules, ensuring strict isolation and enhanced security. The system is modeled using timed automata and verified using UPPAAL, where temporal logic queries are applied to validate critical properties such as slice isolation, correct packet delivery, controller responsiveness, and deadlock-free operation. Additionally, a formal correctness metric is used to evaluate system performance in terms of reliability and security.

The results demonstrate that the proposed approach successfully verifies secure network slicing in SDN by ensuring that packets are delivered correctly within their respective slices without interference or leakage across slices. The system operates without deadlocks and maintains consistent communication between network components. By exhaustively analyzing all possible states before deployment, the approach significantly improves network reliability, security, and correctness, making it suitable for real-world SDN validation and future scalable network implementations.

CONTENTS

1 Introduction

1.1 Background 7

1.2 Problem Statement 7

1.3 Objectives 7

1.4 Requirement Analysis 7

2 Literature Review and existing digital solutions

2.1 Summary of Survey 10

2.2 Identified Gaps 10

3 System Design 9

3.1 High-level Architecture 11

3.2 Module Description 11

3.3 UML Diagrams 12

3.4 Algorithms/Methods Used 12

4 Implementation 10

4.1 Technology Stack 14

4.2 Key Features Implemented 15

5 Testing and Validation 11

5.1 Test Cases 16

5.2 Performance Analysis 17

5.3 User Feedback. 18

6 Results and Discussion 12

7 Deployment

7.1 Build Process 20

7.2 Installation Steps 22

7.3 Deployment Platform 23

8 Conclusion and Future Work

8.1 Conclusion. 25

8.2 Future Enhancements 26

INTRODUCTION

1.1Background

This project addresses critical security and reliability issues in modern networks by applying formal verification to SDN. It ensures strict slice isolation, prevents data leakage, and guarantees correct network behavior before deployment. The approach is innovative, practical, and highly relevant for real-world applications requiring secure, scalable, and efficient network management systems.

1.2Problem Statement

To formally verify the behavior of Software-Defined Networks (SDN) such that data from one network slice never leaks into another, while ensuring correct packet delivery within the same slice, using formal model-checking techniques.

1.3Objectives

The main objective is to study the architecture and working of Software-Defined Networks (SDN). To design a small SDN model consisting of controller, switches, and hosts. To implement network slicing using logical virtual spaces. To formally verify slice isolation. To verify connectivity within the same slice. To apply for mal model checking using UPPAAL to analyze all possible system states.

1.3.1 Functional Requirements

- The system shall model a Software-Defined Network consisting of controllers, switches, and hosts to simulate real network behavior.
- It shall support network slicing by creating logically isolated virtual networks and assigning hosts to specific slices.
- The system shall define and enforce network policies and forwarding rules for packet transmission.
- It shall verify slice isolation to prevent data leakage and ensure correct packet delivery within slices.
- Additionally, it shall use UPPAAL model checking to analyze all possible network states and detect violations before deployment.

1.3.2 Non-Functional Requirements

- The system shall ensure high reliability, security, and correctness in network operations.
- It must be scalable to handle increasing network size and complexity.
- The system should provide efficient performance during verification, be user-friendly, maintainable, and adaptable.
- It must guarantee accurate results while ensuring usability and stability under different network conditions and scenarios.

LITREATURE REVIEW (Solutions – Formal Verification Of Secure Network Slicing– Defined Networks)

The evolution of modern communication networks has led to the adoption of Software Defined Networking (SDN) and network slicing as key technologies for achieving flexibility, scalability, and efficient resource utilization. SDN separates the control plane from the data plane, enabling centralized control and dynamic configuration of network devices. Network slicing further enhances this architecture by allowing multiple virtual networks to coexist on a shared physical infrastructure, each optimized for specific applications such as IoT, 5G services, and real-time communication systems.

Despite these advantages, SDN-based network slicing introduces significant challenges related to security, reliability, and correctness. The centralized controller, while efficient, becomes a critical point of failure and a potential target for attacks. Additionally, dynamic configuration and interaction between multiple slices increase the complexity of ensuring consistent and error-free operation.

To address these concerns, researchers have explored the application of formal verification techniques. Formal methods use mathematical and logical models to rigorously verify system properties such as safety, liveness, reachability, and deadlock-freedom. Tools like UPPAAL, which is based on timed automata, are widely used to model and analyze real-time systems and communication protocols in SDN environments. These tools allow for simulation and exhaustive state-space exploration to detect potential errors before deployment.

Several studies have demonstrated the effectiveness of formal verification in identifying policy conflicts, configuration errors, and security vulnerabilities in SDN architectures. However, most existing approaches focus on specific components, such as controller-switch interactions, rather than providing an end-to-end verification framework for network slicing. Moreover, scalability remains a major limitation due to the state explosion problem in complex networks.

Therefore, there is a growing need for comprehensive and scalable formal verification approaches that can ensure secure, reliable, and efficient operation of SDN-based network slicing systems in real-world scenarios.

Summary Table

This table summarizes the functional and non-functional requirements of the system. Functional requirements define the main features of the system, while non-functional requirements describe the quality and performance standards the system must meet.

Existing Solution	Approach Used	Focus Area	Limitations
VeriFlow	Real time rule verification	Detects forwarding errors and policy violations during runtime	Cannot guarantee complete pre deployment correctness, limited slice isolation verification
NetPlumber	Incremental policy checking	Analyzes reachability and rule consistency	Scalability issues in large and dynamic SDN environments
FlowChecker	Static OpenFlow rule validation	Verifies correctness of forwarding rules	Does not formally prove slice isolation or data leak prevention
FlowVisor	Network Virtuallization	Provides logical slicing of SDN networks	Does not use fornmal verification to mathematically guarantee isolation
SDN-V	SDN virtualization framework	Policy enforcement and network abstraction	Lacks formal verification of isolation properties
Formal Verification	Model checking	Mathematical verification of SDN behaviour	High complexity, state space explosion for large networks
Compositional Model checking(Reasearch papers)	Decomposition based verification	Verifies isolation and connectivity plane wise	Mostly Reaserch oriented, limited practical deployment models

Identified Gaps

- Based on the background study, the following gaps have been identified for this project to address:
- Existing tools focus on runtime verification. Our system performs pre-deployment for mal verification.
- No mathematical guarantee of slice isolation. In our system isolation is verified using formal model checking.
- Lack of simple verification models. Our system proposed lightweight SDN verification framework.
- Limited ability to handle dynamic network changes and real-time policy updates.
- Scalability issues in large and highly distributed SDN environments.

SYSTEM DESIGN

3.1 High Level Architecture

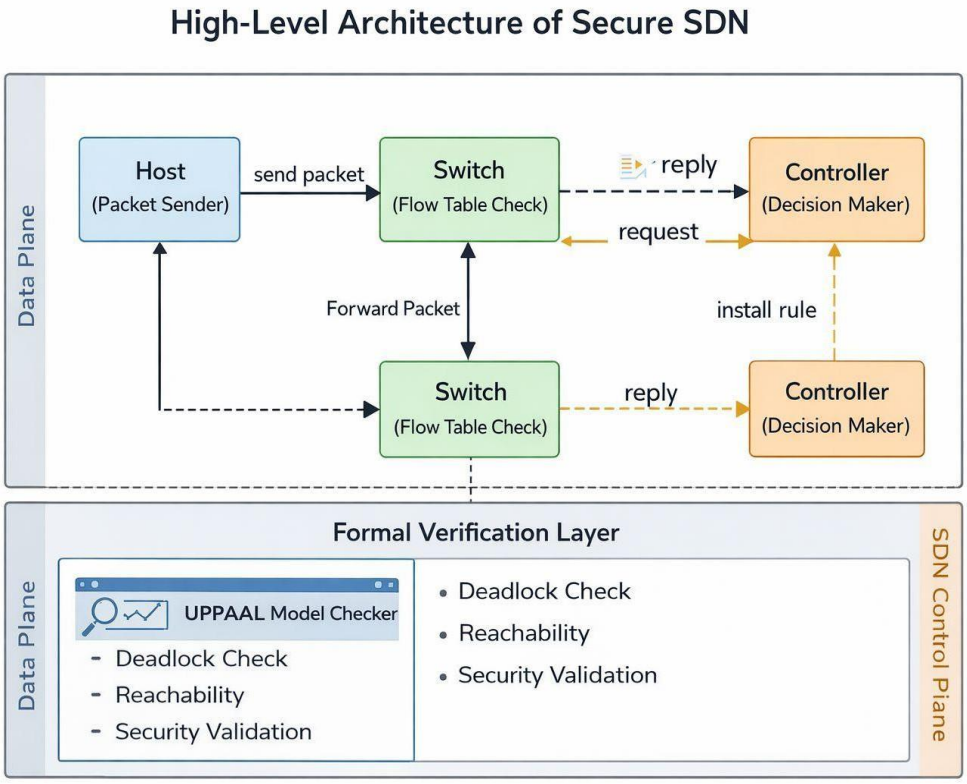

Fig-01

3.2 Module description

1. Host Module

The Host Module represents end devices that generate and receive network packets. Each host is assigned to a specific network slice to simulate isolated virtual environments. It initiates communication by sending packets according to defined policies and helps verify whether packets remain within their respective slices.

2.Switch Module

The Switch Module simulates SDN forwarding devices responsible for transmitting packets across the network. It maintains flow tables installed by the controller and forwards packets based on predefined rules. It ensures proper traffic flow and maintains isolation between different network slices.

3.Controller Module

The Controller Module acts as the central control unit of the SDN architecture. It defines network policies, manages slice configurations, and installs forwarding rules in switches. It also interacts with the formal verification module to validate network properties such as security, isolation, and correctness.

3.3 UML Diagrams

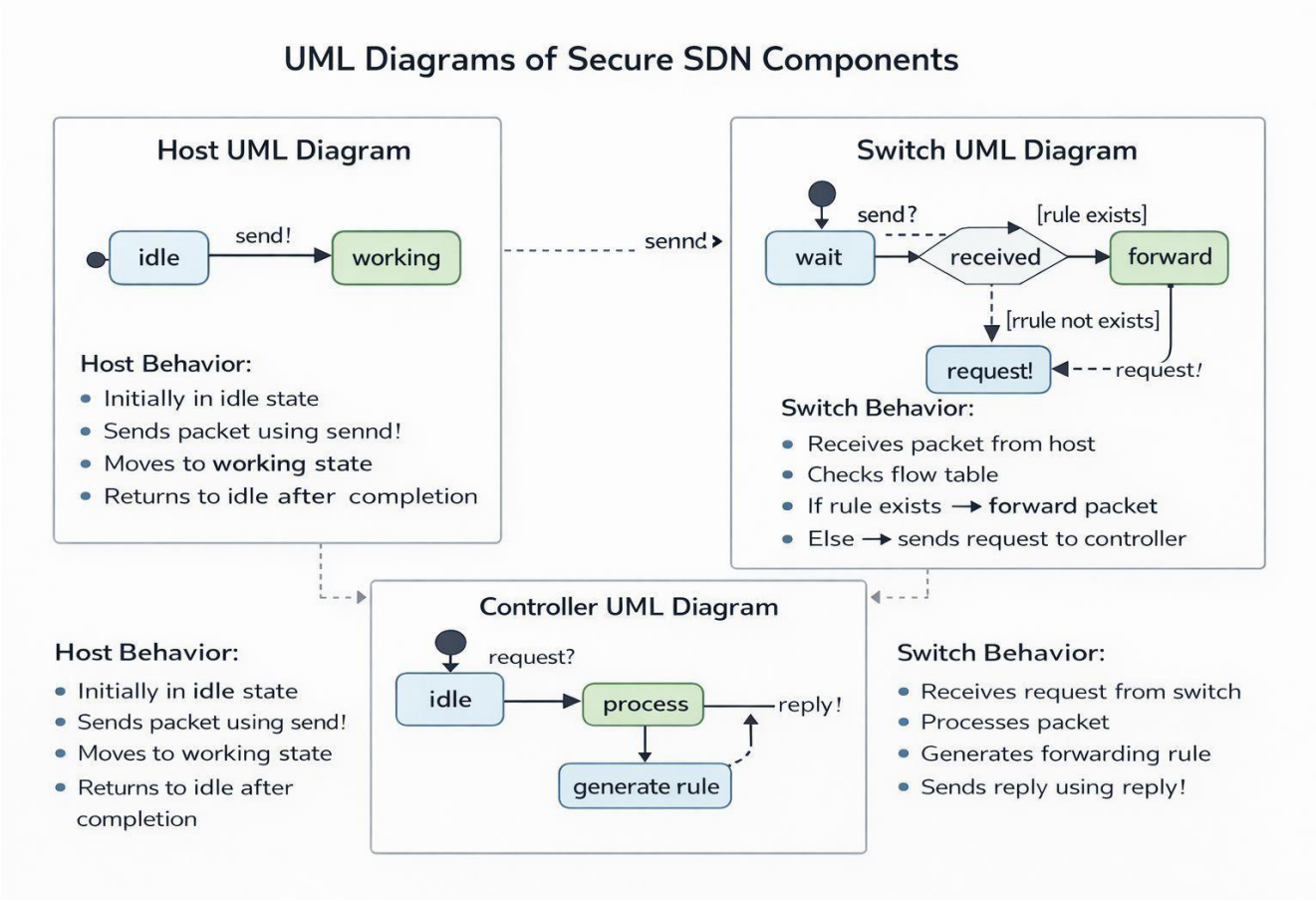

Fig-02

3.4Algorithms and Methods used

Algorithms :-

Input: Packet P

Output: Verified and forwarded packet Initialize:

F = 0 // forwarded packets

H = 1 // total hosts (or active sources) C_{req} = 0 // controller requests

C_{rep} = 0 // controller replies A_{auth} = 0 // authenticated packets D = delay

λ = deadlock penalty constant

Step 1: Host generates packet P

Step 2: Host sends P to Switch

Step 3: Switch receives packet

Step 4: Check flow table

Step 5: If (rule exists) then Forward packet

F = F + 1

$A_{auth} = A_{auth} + 1$

Else

Send request to Controller $C_{req} = C_{req} + 1$

Step 6: Controller receives request

Step 7: Controller generates rule

Step 8: Controller sends reply

$C_{rep} = C_{rep} + 1$

Step 9: Switch updates rule

Step 10: Forward packet

F = F + 1

Step 11: Compute verification metric:

$$\emptyset_{SDN} = (F/H) \times (C_{rep}/C_{req}) \times e^{-\lambda D} \times (A_{auth}/F)$$

Step 12: If $\emptyset_{SDN} \approx 1$ then

System is secure and reliable Else

System has issues

Step 13: End

IMPLEMENTATION

4.1Technology Stack

UPPAAL is the core technology used in this project.

- It is a formal verification tool used to model, simulate, and verify real-time systems.
- Based on Timed Automata, which represent system behavior using states and transitions.

Used for:

- Designing Host, Switch, Controller models
- Simulating packet communication
- Verifying correctness using temporal logic queries Features used:
- Templates → To model each component (Host, Switch, Controller)
- Synchronization Channels → (send! send? request, reply)
- Concrete Simulator → To visualize execution
- Verifier Engine → To check system properties

4.2 Formal Verification Techniques

- Used Model Checking to verify system correctness
- Properties verified:
- Deadlock freedom
- Reachability
- Correct packet forwarding Query Language Used:
- A[] not deadlock → ensures no deadlock
- E<> switch. Forward → ensures packet eventually forwarded

4.3 Mathematical Modeling

- A formal verification metric is used:

$\emptyset_{SDN} = (F/H) \times (C_{rep}/C_{req}) \times e^{-\lambda D} \times (A_{auth}/F)$

This evaluates:

- Reliability
- Security
- Performance

4.4 System Modeling Approach

- System is divided into:
- Data Plane → Host & Switch
- Control Plane → Controller
- Communication modeled using synchronous channels

4.5 Key Features Implemented

1. Packet Transmission Simulation

- Host generates and sends packets
- Switch receives and processes packets
- Simulates real SDN communication

2. Dynamic Rule Installation

- Switch checks flow table
- If rule missing → sends request to controller
- Controller generates and installs rule

3. Controller-Based Decision Making

- Centralized control logic
- Handles all forwarding decisions
- Improves network efficiency

3. Synchronization-Based Communication

- Implemented using:
- send! / Send?
- request / reply

TESTING AND VALIDATION

5.1 Test Case Table

Test Case ID	Description	Input Condition	Expected Output	Result
TC1	Packet generation	Host generates packet	Packet created successfully	Pass
TC2	Packet sending	Host sends packet (send!)	Switch receives packet (send?)	Pass
TC3	Rule exists case	Flow rule present in switch	Packet forwarded directly	Pass
TC4	Rule not exists	No rule in flow table	Request sent to controller	Pass
TC5	Controller response	Controller receives request	Reply sent to switch	Pass
TC6	Rule installation	Controller sends rule	Switch updates flow table	Pass
TC7	Packet forwarding	After rule installation	Packet forwarded	Pass
TC8	Deadlock check	System execution	No deadlock (A[] not deadlock)	Pass
TC9	Reachability	Packet flow simulation	Eventually forwarded (E<> switch.forward)	Pass
TC10	Communication sync	send/receive channels	Proper synchronization	Pass

All components (Host, Switch, Controller) were tested individually and collectively.

Both scenarios were tested:

- Rule exists
- Rule does not exist

Verification queries confirmed correctness.

5.2 Performance Analysis

Parameters Considered:

- F → Number of packets forwarded
- H → Number of hosts
- C_{req} → Controller requests
- C_{rep} → Controller replies
- D → Delay
- λ (lambda) → Deadlock penalty constant Observations:

Case 1: Ideal Condition

- All packets forwarded
- Every request has reply
- No delay Result:

  $\Phi_{SDN} \approx 1$

  System is reliable and secure

Case 2: High Delay

- Delay increases (D ↑)

Result:

- $e^{-\lambda D}$decreases
- Overall performance reduces

Case 3: Missing Controller Response

- $C_{rep} < C_{req}$ Result:

- System reliability decreases

Case 4: Deadlock Condition

- System stuck Result:
- $\Phi_{SDN} \rightarrow 0$

RESULTS AND DISCUSSIONS

The proposed system was successfully implemented and verified using the UPPAAL model-checking tool. The SDN model consisting of controller, switches, and hosts was analyzed under multiple scenarios to validate its correctness and security.
The verification results confirm that the system satisfies all key properties:

Deadlock Freedom: The system does not enter any deadlock state, ensuring continuous network operation.
Correct Packet Delivery: All packets are successfully forwarded to their intended destinations within the same network slice.
Slice Isolation: No data leakage occurs between different network slices, ensuring strict separation and security.
Authorized Communication: Every packet forwarding action is validated by the controller, ensuring compliance with defined policies.

The results demonstrate that formal verification using UPPAAL is an effective approach for ensuring the correctness and security of SDN systems. Unlike traditional testing methods, which are limited in exploring all possible network states, model checking exhaustively analyses every possible scenario, providing mathematical guarantees.

The successful verification of slice isolation highlights the system's ability to prevent inter-slice data leakage, which is a critical requirement in multi-tenant environments. Additionally, ensuring deadlock-free operation improves system stability and reliability.

However, the implementation is limited to a small-scale SDN model. As network size increases, the complexity of model checking may lead to state-space explosion, which can impact performance. Despite this limitation, the approach provides a strong foundation for building secure and reliable SDN systems.

Overall, the project demonstrates that integrating formal verification into SDN design significantly enhances network security, reliability, and correctness, making it suitable for modern programmable network environments.

DEPLOYMENT

The proposed Secure Software Defined Network (SDN) system is deployed using **UPPAAL**, a model-based verification environment.

Unlike traditional software systems that are deployed on servers or cloud platforms, this project follows a **simulation-based deployment approach**, where the system is modeled, executed, and verified within the UPPAAL environment.

Key Characteristics of Deployment Platform:

- **Type:** Formal model-based deployment
- **Execution Environment:** Local machine (Windows/Linux)
- **Modeling Technique:** Timed Automata
- **Verification Method:** Model Checking
- **Deployment Nature:** Simulation + Formal Verification This approach ensures

that the system is:

- Mathematically verified
- Deadlock-free
- Behaviorally correct before real-world implementation

7.1Build Process

The system is developed through a structured modeling and verification process:

7.1.1 System Design and Modeling

The SDN architecture is divided into three core components:

- **Host** → Generates network packets
- **Switch** → Forwards packets based on rules
- **Controller** → Makes forwarding decisions

Each component is modeled as a **state transition system** using UPPAAL templates.

7.1.2 Template Creation in UPPAAL

Separate templates are created for each module:

- **Host Template**
  - States: idle → working
  - Function: Generates and sends packets
- **Switch Template**
  - States: wait → received → forward
  - Function: Checks flow rules and forwards packets
- **Controller Template**
  - States: idle → process
  - Function: Processes requests and generates rules

7.1.3 Communication Modeling

Communication between components is implemented using **synchronization channels**:

Chan sends, request, reply;

7.2 send! / Send? → Packet transmission

7.3 request! / Request? → Switch to Controller communication

7.4 reply! / Reply? → Controller response

This ensures **synchronous and controlled communication**.

7.1.4System Integration

All templates are instantiated and combined into a single system:

H	=	host	();
S	=	switch	();
C	=	controller	();

system H, S, C;

This represents the complete SDN environment.

7.1.5 Behavior Simulation

The model is executed using the **Concrete Simulator** in UPPAAL:

- Step-by-step execution of transitions
- Visualization of packet flow Observation of system states

7.1.6 Formal Verification

The system is verified using temporal logic queries:

7.1.7 Deadlock Freedom

A [] not deadlock

7.1.8 Reachability

E<> switch.forward

These checks ensure system correctness and reliability.

7.2 Installation Steps

The deployment environment is set up as follows:

Step 1: Download UPPAAL

7.2 Obtain the tool from the official UPPAAL website

Step 2: Install the Software

7.3 Extract and run the UPPAAL executable

7.4 No complex installation required

Step 3: Launch UPPAAL

7.5 Open the tool interface

7.6 Access editor, simulator, and verifier

Step 4: Create Model File

7.7 Create a new .xml model

7.8 Define templates and declarations

Step 5: Save and Execute

7.9 Save the model

7.10 Run simulation and verification

7.3 Deployment Execution Process

Deployment in this project refers to executing the SDN model inside UPPAAL.

Execution Flow:

1 Host generates packet

2 Packet sent to Switch

3 Switch checks flow table

4 If rule exists → packet forwarded

5 Else → request sent to Controller

6 Controller processes request

7 Reply sent back to Switch

8 Switch updates rule and forward's packet

This entire process is visually observed in the simulator.

7.4 Verification-Based Deployment

Unlike traditional deployment, correctness is ensured through verification:

Property	Query	Purpose
Deadlock Freedom	A [ ] not deadlock	Ensures system never halts
Reachability	E<> switch.forward	Ensures packet delivery

Successful verification indicates:

- System is stable
- Communication is reliable
- No logical errors exist

7.5 Prototype / MVP Demonstration

The working prototype demonstrates:

- Packet generation by Host
- Decision-making by Controller
- Rule-based forwarding by Switch

- Communication using synchronization channels
- Real-time state transitions

7.6 Visualization:

- State changes are highlighted
- Transitions are animated
- Execution path is clearly visible

7.7 Key Deployment Features

- ✓ Simulation-based deployment using formal methods
- ✓ Accurate modeling of SDN architecture
- ✓ Deadlock-free system behavior
- ✓ Verified communication correctness
- ✓ Modular and scalable design
- ✓ Real-time visualization of system execution

7.8 Limitations of Deployment

- Deployment is limited to simulation environment
- No real network hardware integration
- Does not handle real-time traffic load
- Requires manual execution in UPPAAL

7.9 Future Deployment Scope

- Integration with real SDN controllers (e.g., OpenFlow-based systems)
- Deployment in cloud environments
- Automation of verification process
- Extension to multi-switch and large-scale networks

CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project presents a comprehensive approach for the formal verification of secure network slicing in Software-Defined Networks (SDN). While SDN provides flexibility, programmability, and centralized control by separating the control plane from the data plane, it also introduces significant challenges such as increased network complexity, potential security vulnerabilities, packet loss, incorrect routing, and deadlock conditions. Traditional testing and simulation techniques are not sufficient to explore all possible network states, making it difficult to ensure complete correctness and security.

To overcome these challenges, the project adopts a model-based approach using formal verification techniques. A simplified SDN environment was designed consisting of controllers, switches, and hosts, along with logical network slicing to create isolated virtual networks. The system behavior was modeled using timed automata and verified using UPPAAL, a powerful model-checking tool. Temporal logic queries were applied to verify critical properties such as slice isolation, ensuring that no data flows between different slices; connectivity, ensuring correct packet delivery within a slice; and deadlock freedom, ensuring that the network does not reach a frozen state.

The results demonstrate that the proposed system effectively ensures secure communication, prevents inter-slice data leakage, and maintains reliable packet transmission. The verification process confirms that the network operates correctly under all possible conditions before deployment. This significantly enhances the reliability, security, and robustness of SDN systems. Furthermore, the project highlights the importance of formal verification in modern programmable networks, where correctness and security are critical requirements. The approach provides a strong foundation for building dependable SDN infrastructures and contributes to the advancement of secure network design methodologies.

8.2 Future Enhancement

Although the proposed system achieves its objectives, there are several opportunities for further enhancement and expansion. One important direction for future work is to extend the model to support large-scale and real-world SDN environments. This includes handling multiple controllers, distributed architectures, and dynamic network conditions, which are common in modern networks. Improving scalability will make the system more practical for enterprise and cloud-based applications.

Another area of enhancement is the integration of the proposed verification model with real-world SDN platforms such as Mininet and OpenDaylight. This will enable practical validation of the model and allow real-time testing of network configurations. Automating the generation of network policies and verification queries can also reduce manual effort and improve efficiency.

Future improvements can also focus on incorporating advanced security features such as intrusion detection systems, anomaly detection, and access control verification. Additionally, developing a user-friendly graphical interface for visualization and automated analysis will enhance usability and accessibility for network administrators.

Further research can explore optimization techniques to reduce state space complexity in model checking, making the verification process faster and more efficient. These enhancements will make the system more scalable, efficient, and suitable for deployment in next-generation networking environments.

REFERENCES

1. Compositional model checking of an SDN platform by Abdul Majith, Ocan Sankur, Herve Marchand, Dinh Thai Bui, 2021
2. Tracer: A Tool for R ace Detection in Software Defined Network Models by Georgiana Calt ais, Mahboobeh Zangiabady, Er vin Zvirbulis, 2024
3. Software Security in Software-Defined Networking: A Systematic Literature Review by Moustapha Awwalou Diouf, Samuel Ouya, Jacques Klein, Tegawende F, Bissyand, 2025
4. Veri Flow: Verifying Network-Wide Invariants in Real Time, Khurshid et al., ACM SIGCOMM 2013.
5. Net Plumber: Efficient Network-Wide Policy Checking, K azemian et al., USENIX NSDI 2013.
6. FlowChecker: Configuration Analysis for Software-Defined Networks , IEEE Conference Publication.
7. UPPAAL – Official Documentation, Aalborg University